

Jour 1 : L'interface TUN/TAP et Ethernet

- **Pratique** : Configuration du périphérique virtuel sous Linux. Lecture des premières trames Ethernet brutes.

Passer en mode sudo ou juste utiliser sudo à chaque commande

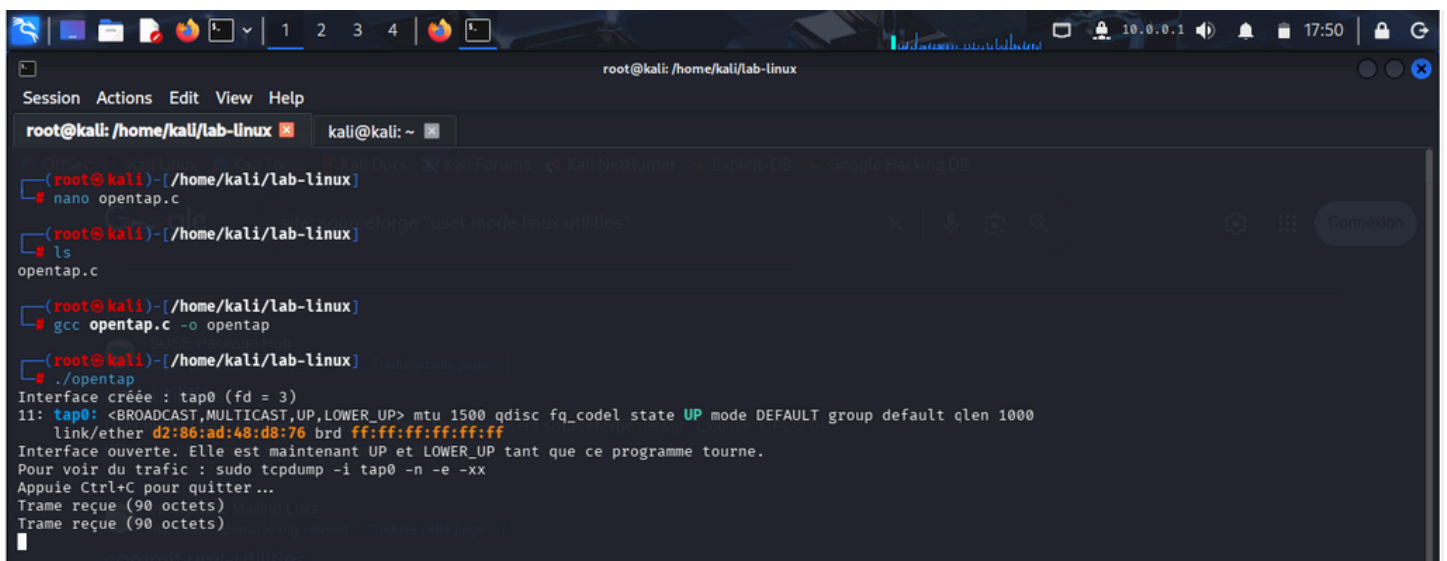
```
sudo ip tuntap add dev tap0 mode tap user $USER
```

```
sudo ip link set tap0 up
```

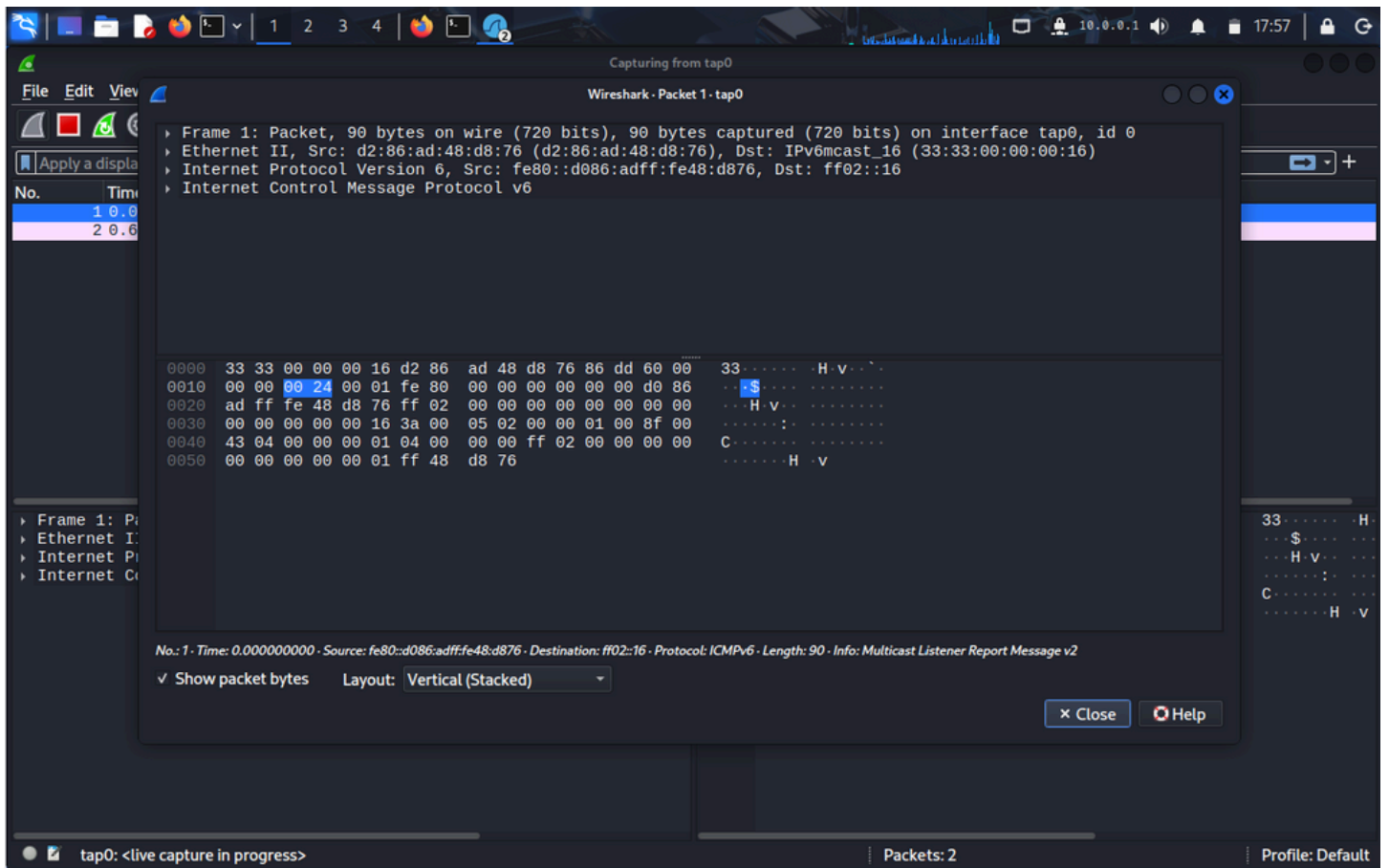
```
sudo ip addr add 10.0.0.1/24 dev tap0
```

en suite on vérifie avec un ip a et on voit que l'interface tap0 est bien là mais administrativement up mais il est opérationnellement down car aucun processus n'est ouvert sur ce interface donc on va le forcer à s'ouvrir pour pouvoir l'avoir sur wireshark

on utilisera le script tiré de la page et on y rajoute un main puis on le compile et on le lance



```
root@kali: /home/kali/lab-linux
Session Actions Edit View Help
root@kali: /home/kali/lab-linux x kali@kali: ~
nano opentap.c
ls
opentap.c
gcc opentap.c -o opentap
./opentap
Interface créée : tap0 (fd = 3)
11: tap0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether d2:86:ad:48:d8:76 brd ff:ff:ff:ff:ff:ff
Interface ouverte. Elle est maintenant UP et LOWER_UP tant que ce programme tourne.
Pour voir du trafic : sudo tcpdump -i tap0 -n -e -xx
Appuie Ctrl+C pour quitter...
Trame reçue (90 octets)
Trame reçue (90 octets)
```



Sur **wireshark** on a bien deux trames : en les analysant on voit que l'adresse MAC source est la même que celle de l'interface tap0 et la destination semble être une adresse MAC multicast au format ipv6. D'autres trames sont envoyées fréquemment.

- **Focus Sécurité** : Comprendre l'isolation entre le Kernel et l'User-space.